

Ride-Sharing Platform

An on-demand marketplace matching riders with nearby drivers in real time — dispatch, live location, dynamic pricing, and payments at scale.

SCOPE Rider • Driver • Marketplace TARGET Senior system-design interview

SCALE 100M users • 10M rides/day

00 Problem & scope

Design the backend for an on-demand ride-hailing service. A **rider** requests a trip between two points; the system finds the best nearby **driver**, negotiates the offer, tracks the trip in real time, computes a fare, and settles payment. The hard parts are the **real-time geospatial workload** (millions of location writes/sec), **low-latency matching**, and **financial correctness** under partial failure.

IN SCOPE

- Ride request, matching & dispatch
- Real-time driver location & tracking
- Trip lifecycle & state management
- Dynamic pricing / surge
- Payments & the money ledger
- ETA / routing & notifications

OUT OF SCOPE

- Pool / shared rides & carpool routing
- Food / freight verticals
- Fraud & risk modeling internals
- Driver onboarding / background checks
- Ratings & reputation algorithms

01 Requirements

Functional

ACTOR	CAPABILITIES
Rider	See nearby drivers & ETA · request a ride (pickup/dropoff) · get a fare estimate · track the assigned driver live · pay automatically · rate the trip.
Driver	Go online/offline · stream location · receive & accept/decline offers · navigate to pickup and dropoff · complete trip · receive earnings.
System	Match rider→best driver · compute price + surge · run the trip state machine · settle payments · send push/real-time updates.

Non-functional

Low latency Match < 1–2 s; nearby query p99 < 200 ms; location fanout near-instant.	High availability Marketplace must stay up; degrade gracefully, never lose a trip or a payment.	Consistency Strong for trips & money; eventual for location & analytics is fine.
Scale & geo 10s of millions DAU, multi-region, sharded by geography.	Durability Trips & ledger entries survive any single-node/AZ loss; audited.	Real-time write load Sustained ~250K location writes/s — the dominant scaling constraint.

02 Capacity estimation

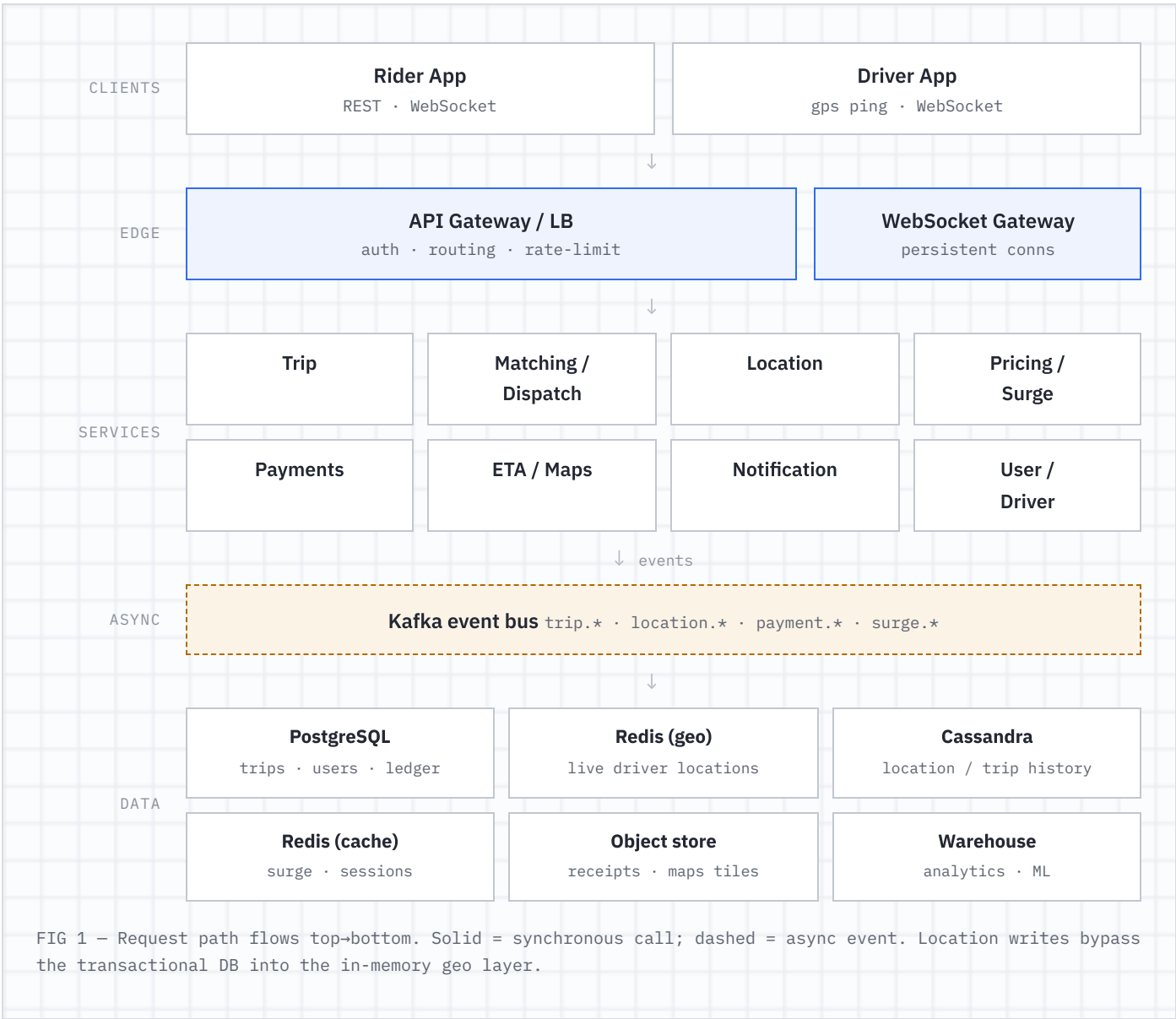
ASSUMPTIONS 100M registered users · 5M daily active riders · 3M drivers, ~1M online concurrently at peak · 10M completed rides/day · driver location ping every 4 s.

~575/s Peak ride requests 115/s avg × 5 peak	~250K/s Location writes 1M drivers ÷ 4 s	~3.6TB/yr Trip records 10M/day × 1 KB	~2TB/day Raw location stream TTL'd / downsampled
---	---	--	---

READ THE NUMBERS

Ride requests are trivial (~575/s). The system is defined by the **location write firehose**: ~250K writes/s, ~25 MB/s ingest, and if stored raw ~2 TB/day. This is **write-heavy, ephemeral, and geo-partitioned** — it belongs in memory (Redis / in-process geo index), not in your transactional DB. Persisting every ping to Postgres would melt it. That single observation drives most of the architecture.

03 High-level architecture



04 Core API surface

VERB	ENDPOINT	PURPOSE
GET	/v1/drivers/nearby?lat&lng	Nearby cars + ETA for the map view
POST	/v1/fare/estimate	Quote fare + current surge for a route
POST	/v1/trips	Request a ride (Idempotency-Key)
GET	/v1/trips/{id}	Trip state + assigned driver + live ETA
POST	/v1/trips/{id}/cancel	Cancel (may incur fee)
POST	/v1/offers/{id}/accept	Driver accepts a dispatch offer
WS	/v1/driver/location	Location ping stream (driver→server)
WS	/v1/trips/{id}/track	Live driver position (server→rider)

All mutating calls take an Idempotency-Key so a retried request never creates a duplicate trip or charge. High-frequency location and tracking traffic uses **WebSockets**, not REST, to avoid per-message HTTP overhead.

05 Data model

Split by access pattern and consistency need. **Trips, users and the ledger** are relational (strong consistency, joins, transactions). **Live location** is key-value in memory. **Location & trip history** is wide-column (append-heavy, time-series).

trips · PostgreSQL

```
id uuid pk
rider_id · driver_id fk
state enum
pickup / dropoff geog
fare_quote_id · surge_mult
requested_at · completed_at
```

driver_status · Redis

```
driver_id key
lat · lng · heading
state online|offline|on_trip
geohash · updated_at
GEOADD drivers:{cell}
```

ledger_entries · PostgreSQL

```
id pk · trip_id
account rider|driver|platform
direction debit|credit
amount · currency
psp_ref · created_at append-only
```

trip_events · Cassandra

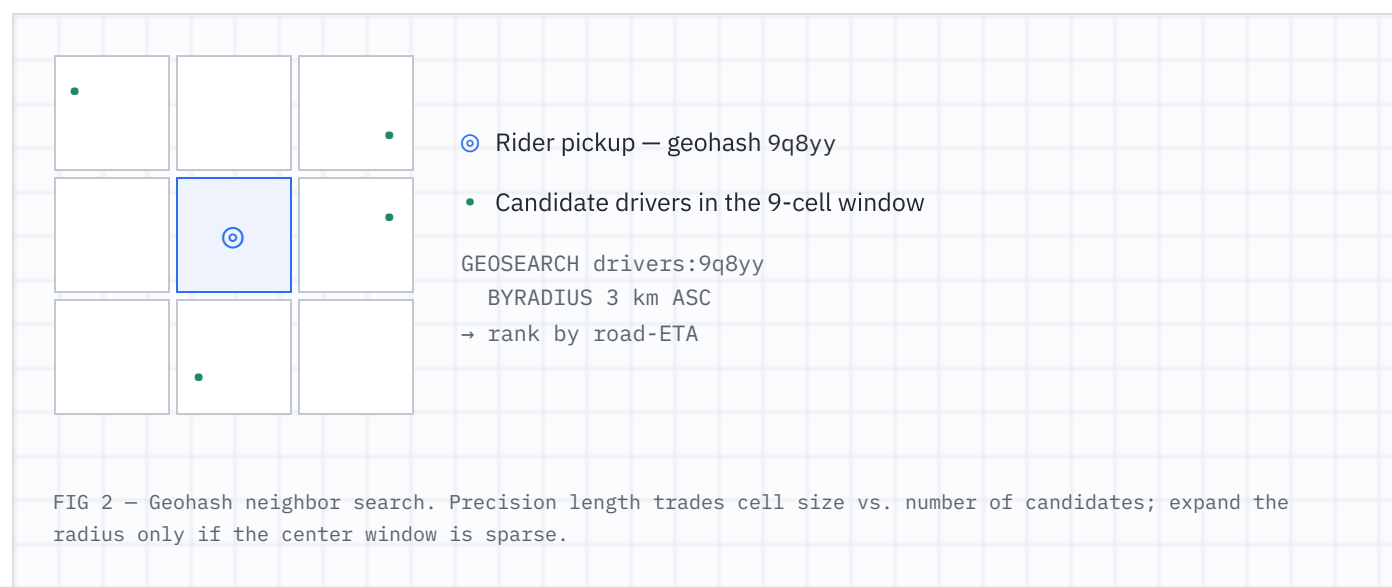
```
trip_id partition
ts clustering
event_type · payload
append-only event log
→ rebuild trip state
```

06 Deep dives

6.1 Real-time location & geospatial indexing

Drivers stream a GPS ping every few seconds over a persistent WebSocket to the **Location service**, which (a) updates the driver's position in the in-memory **geo index** and (b) publishes to Kafka `location.updates` for downstream consumers. Raw pings are **never written to the transactional DB** — they're kept in Redis with a short TTL and sampled into Cassandra for history/analytics.

The index buckets drivers by **geohash / S2 cell**. A "nearby drivers" query computes the pickup's cell and reads that cell plus its 8 neighbors — bounding the search to a handful of cells instead of scanning millions of drivers. Cells are the shard key, so the whole geo layer partitions naturally by geography.



6.1.1 Choosing a spatial index

All three viable options turn 2-D coordinates into a 1-D key so points can be range-scanned and bucketed. They differ in how they handle non-uniform density — which matters a lot when drivers cluster downtown and thin out in suburbs.

SCHEME	IDEA	TRADEOFF
Geohash	Interleave lat/lng bits into a base-32 string; shared prefix ⇒ spatial proximity.	Dead simple, string-prefix queries, native in Redis. Fixed grid ignores density.
QuadTree	Recursively split a cell into 4 once it holds too many points.	Adapts to density (deep downtown, shallow rural). In-memory tree needs rebalancing.
S2 (Hilbert)	Project the sphere onto a cube, order cells along a Hilbert curve.	Best locality, no polar distortion, 30 levels. More machinery than most designs need.

PICK

Geohash on Redis for the live index — it ships with GEOADD/GEOSearch, so there's no custom tree to operate, and a fixed grid shards cleanly by prefix. Reach for a QuadTree/S2 only if uneven density makes fixed cells wasteful.

6.1.2 Precision & cell size

Geohash length sets cell size. Too coarse and each cell holds thousands of drivers to rank; too fine and a normal-radius search must union dozens of cells. For city dispatch, precision **6** (~1.2 km) is the sweet spot, dropping to **5** (~5 km) for sparse regions.

PRECISION	≈ CELL SIZE	USE
5	5 km × 5 km	Sparse / rural regions
6	1.2 km × 0.6 km	Default city dispatch
7	150 m × 150 m	Dense cores, precise pickup

6.1.3 The cell-boundary problem

The nearest driver may sit just across a cell edge — physically 50 m away but in a different geohash. Querying only the pickup's cell would miss them. This is exactly why the search reads the center cell **plus its 8 neighbors** (FIG 2): the 3×3 window guarantees any driver within the cell's dimension is considered, then results are filtered by true distance and ranked by road-ETA. If the window comes back sparse, widen by one geohash character (a coarser cell) rather than adding rings.

6.1.4 Write & read path

Write (ping, ~250K/s)	Read (nearby query)
-----------------------	---------------------

1. Driver WS ping → Location service.
2. Compute geohash; GEOADD updates position in place (idempotent, last-write-wins).
3. Emit `location.updates` to Kafka for tracking fanout + history.
4. No DB write; entry carries a TTL so a crashed driver ages out.

1. Geohash the pickup point.
2. GEOSEARCH center + 8 neighbors by radius.
3. Drop on-trip / offline drivers, filter by true distance.
4. Hand candidates to Matching for road-ETA ranking (§6.2).

SHARDING The index is partitioned by cell prefix (region), so a city's drivers live on one shard and a dispatch query stays local. A **hot cell** — airport, stadium — is handled by promoting it to a finer precision (splitting one bucket into many) and load-shedding read frequency, so a single Redis key never becomes the bottleneck.

6.2 Matching & dispatch

- 1 Rider requests; Matching service pulls candidates from the geo index (§6.1).
- 2 Rank by **road-network ETA** (not straight-line), plus accept-rate & rating.
- 3 Offer to the top driver; place a short-TTL **lock** on driver + request.
- 4 On accept → create trip & assign. On decline/timeout → next candidate.

CORRECTNESS — NO DOUBLE-BOOKING

Two riders must never be assigned the same driver. Guard with an **atomic conditional claim**: a Redis lock `SET driver:{id} trip:{id} NX EX 15` gates the offer, and the DB enforces a partial-unique constraint of one active trip per driver. The lock TTL auto-recovers if a dispatcher crashes mid-offer.

6.2.1 Ranking candidates

The geo query (§6.1) returns nearby drivers; the ranker picks who to offer first. "Closest" is the wrong signal — a car 300 m away across a river is worse than one 800 m down a straight road. Score by **road-network ETA to pickup**, then bias toward drivers likely to accept and complete cleanly:

```
score = w1·pickup_eta - w2·accept_rate - w3·rating + w4·idle_penalty
```

The `idle_penalty` is a fairness lever — surface drivers who've waited longest so earnings spread evenly. Weights are tuned offline against completion rate and rider wait time.

6.2.2 Dispatch strategy: greedy vs. batch

Greedy sequential offer

Offer to the single best driver; on decline/timeout, move to the next. Simple, low-latency, locally optimal — the default.

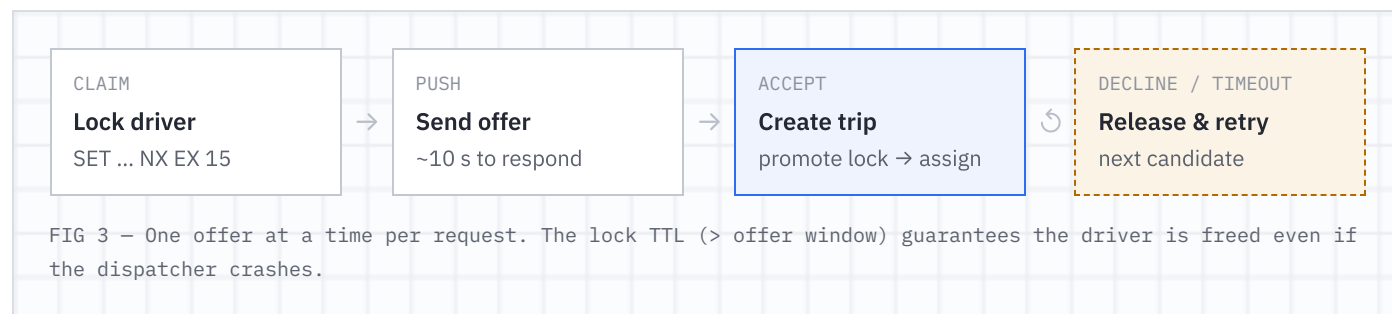
Batch matching (windowed)

Collect requests over a short window (~a few s) and solve the assignment globally (min-cost bipartite). Better fleet-wide ETAs; adds a bit of wait.

WHY BATCH WINS IN DENSE CELLS

Greedy can hand the one nearby car to request A while request B (who needed it more) waits. Solving a whole window at once avoids these collisions and lowers *average* pickup time — worth the few-second delay exactly where supply is tight. Fall back to greedy in sparse cells where there's nothing to optimize.

6.2.3 The offer & lock protocol



6.2.4 Failure & edge handling

No drivers in window	Widen the geohash radius one step; if still empty, queue the request and notify the rider of a wait.
Serial declines	Cap retries, then broaden radius or raise the offer (surge) to widen the acceptable pool.
Dispatcher crash	Lock TTL expires → driver auto-released; request is idempotent (Idempotency-Key) so a retry can't double-assign.
Driver cancels post-accept	Return trip to MATCHING, re-dispatch, and record the cancellation against the driver's accept-rate.
Stale location	Skip candidates whose last ping is older than the TTL — they may be offline or in a dead zone.

6.3 Trip lifecycle state machine

The Trip service is the source of truth. Every transition is validated server-side and appended to `trip_events` (event-sourced), so state is auditable and rebuildable, and each event fans out on Kafka to pricing, payments, and notifications.



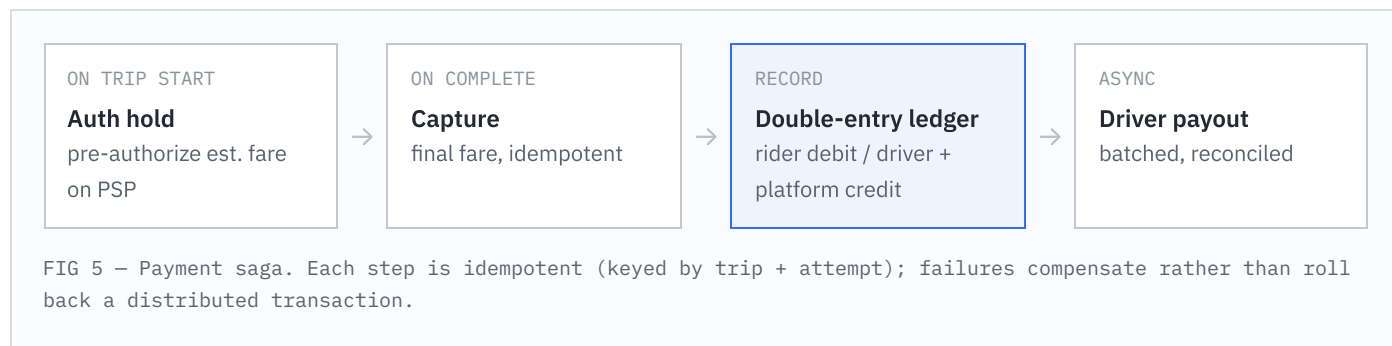
6.4 Dynamic pricing & surge

Fare is computed at quote time and **locked** so the rider isn't repriced mid-request:

```
fare = (base + per_min·mins + per_mile·miles) × surge(cell) + fees
```

A streaming job keeps a **demand ÷ supply ratio per geohash cell** — open requests vs. available drivers — updated every few seconds from Kafka and cached in Redis. High ratio → multiplier > 1, which both rations demand and pulls drivers toward the hot cell. The multiplier is read at quote time and frozen onto the trip.

6.5 Payments & the money ledger



The **ledger is append-only and double-entry** — every cent is a balanced pair of entries, so balances are always reconstructable and auditable. The external PSP (e.g. Stripe/Braintree) holds card data (PCI scope stays out of our system). A **saga** coordinates trip-complete → capture → ledger → payout; a nightly job reconciles our ledger against PSP settlement reports.

6.6 Maps, ETA & routing

A routing engine over a **road-network graph** (e.g. OSRM / a maps provider) computes route + ETA using live traffic and historical speed models. ETA feeds three places: nearby-driver ranking (§6.2), the rider's live countdown, and fare distance/time. Map tiles and common route segments are cached; ETA is the accuracy driver behind good matching, so it's worth the compute over cheap straight-line distance.

6.7 Notifications & real-time delivery

Two channels. **Push (APNs / FCM)** carries discrete events — offer received, driver arrived, receipt — and reaches backgrounded apps. **WebSockets** (via the WS gateway) carry the high-frequency live stream: the driver's moving dot fanned out to the tracking rider. The Notification service subscribes to Kafka trip/location events and routes each to the right channel; a connection registry maps user → gateway node for targeted fanout.

07 Storage choices

DATA	STORE	WHY
Trips, users, ledger	PostgreSQL	ACID transactions, joins, financial correctness; shard by region.
Live driver locations	Redis (GEO)	In-memory, geo-index ops, TTL; absorbs the 250K writes/s.
Location & trip history	Cassandra	Write-optimized, time-series, linear scale-out, TTL.
Surge, sessions, locks	Redis	Sub-ms reads, atomic ops for dispatch locks.
Events between services	Kafka	Durable, replayable, partitioned by geo; decouples producers.
Receipts, map tiles	Object store	Cheap blob storage behind a CDN.
Analytics & ML features	Warehouse	Offline; fed from Kafka, no impact on serving path.

08 Scalability & bottlenecks

Location write firehose

Keep pings in memory; batch & sample to disk; never touch the SQL path. This is the #1 constraint.

Hot cells (airport, event)

Demand spikes in one geohash: surge rations it, split busy cells, and apply backpressure to queries.

Geo-sharding

Partition dispatch, geo index & Kafka by city/region — a trip is local, so cross-shard traffic is rare.

WebSocket connections

Millions of persistent conns → many stateless gateway nodes + a user→node registry; scale horizontally.

Read scaling

Cache surge/ETA; read replicas for trip lookups; the hot path barely hits primary Postgres.

Multi-region

Dispatch runs region-local for latency; user/ledger data has a home region with async replication.

09 Key tradeoffs & follow-ups

SQL for the ledger	Give up NoSQL's easy scale to get transactional guarantees where money demands them. Location data goes the other way.
---------------------------	--

Consistency in dispatch	Brief locks add latency but prevent double-booking. Lock TTLs bound the failure blast radius.
Push vs. poll	WebSockets cost persistent state on the server but crush the latency & overhead of polling for live tracking.
Road ETA vs. haversine	Straight-line distance is free but wrong near rivers/highways; road-network ETA is the cost of good matches.

LIKELY FOLLOW-UPS batch matching vs. greedy dispatch · driver incentives & repositioning · pooled/shared rides · fraud & GPS spoofing · exactly-once payment semantics · region failover & disaster recovery.